

One-page note — Kill the Quote Spreadsheet

What this is: A short note for anyone reviewing the live demo. **Live demo:** <https://kill-the-quote-spreadsheet-lac.vercel.app/> **Code:** <https://github.com/Anirudhmen94/Aerchainfinal>

What we are doing

Procurement buyers still spend days copying vendor quotes into Excel. They send an RFx to five suppliers and get back messy replies — Excel that ignores the template, a PDF with a discount buried in a footnote, a Word paragraph, a phone photo of a rate card, or a one-line email. Then a manager asks one “what if” question and another afternoon disappears.

This prototype ends that week. A buyer drafts the RFx with an AI co-pilot, vendors reply however they like, the system reads every format into one side-by-side comparison (same lines, same units, same currency), and the buyer asks questions in plain English through to an award decision they can defend.

Demo size: corrugated packaging for a Chakan snacks plant · **5 vendors** · **30 line items** · **8 quality questions**.

We define the product as an **evidence-backed procurement decision system**: establish **comparability before price**, separate quoted from normalized values, distinguish eligible vs merely cheap vendors, flag gaps, and keep evidence behind every recommendation.

Who the main users are

User	Why they matter
Category buyer (primary)	Drafts the RFx, reviews the comparison, asks “what if,” and makes the award they must defend.
Category / sourcing manager	Needs visibility when a buyer wants a failing or incomplete vendor on a line — gets a notify in Outbox (not a fake Approve screen) and can review the Audit trail.

Main features we are building

The workspace is free tabs — **Draft** → **Outbox** → **Inbox** → **Compare** → **Ask** → **Award** → **Audit** — not a locked wizard.

- 1. AI-assisted RFx draft** — Plain-language brief becomes ~30 line items, an 8-question quality questionnaire (including knockouts), and five vendors the buyer can edit.
- 2. Outbound stubs (Outbox)** — Preview RFQ cover emails and “send” as files (no real mail server). Same ledger later holds award notices and manager alerts.
- 3. Any-format inbox + parse** — Seed or upload five ugly reply types (Excel off-template, PDF with footnote discount, Word in USD, angled photo, one-line email) and parse them into structured quotes.
- 4. Evidence-backed comparison** — One INR side-by-side matrix. Click a cell to open the **original source**. Unclear values show as uncertain/missing — never silent false precision. FX and unit conversion run in code, not in the model.
- 5. Natural-language Ask** — Chat over the live comparison (e.g. cheapest split among vendors who cleared quality). Answers use extracted data; no hardcoded scripts.
- 6. Quality-gated award** — Pick a vendor per line; failing/incomplete vendors stay off the default list. Choosing one notifies the manager via Outbox.
- 7. Audit trail** — Saves, notices, and overrides are logged so the decision history is reviewable.

How we measure success

Metric	What “good” looks like
Ugly-edge coverage	All five seeded reply formats land in the comparison without forcing a vendor template.
Normalization honesty	USD / odd units show as converted (or flagged when unsafe); partial quotes show as missing (e.g. ~27/30 lines).
Evidence & trust	Every priced cell opens source-of-truth evidence; Audit shows saves, notices, and overrides.
Eligible ≠ cheapest	Knockout-fail vendors stay off the default award list; partial/override creates a manager notify, not silent eligibility.
Analyst defensibility	VP-style / split questions return computed tables + caveats; free-ask is not canned.
Session durability	A cold reload can restore the same event (Blob-backed snapshot).
Buyer time (thesis)	A 5×30 event becomes one live workspace session instead of a retype week + VP afternoon.

What we deliberately left out (and why)

- **Real email (SMTP/IMAP).** Outbox saves stub files only — prove the workflow without building a mail product.
- **Freeze / Unfreeze on Award.** Removed on purpose; save + Audit is enough for a demo.
- **Vendor portal / forced template.** Vendors stay free; we absorb the mess.
- **Approve / Reject UI on Award.** Manager notify-only via Outbox; buyer owns the screen.
- **ERP, payments, POs.** Scope stops at a defensible award.
- **Multi-user login / permissions.** Single-buyer demo so messy replies stay center stage.
- **Hardcoded Ask answers.** Would fake the reasoning the assignment forbids.
- **Silently applying footnote discounts into award totals.** Discounts stay visible as caveats.
- **One-click “run everything” on Home.** Reviewer walks the real path tab by tab.

Where the more interesting problem actually is

The more interesting problem is not the spreadsheet itself. The spreadsheet is where procurement teams manually consolidate a deeper problem: **supplier responses are not immediately decision-comparable**.

A price is only useful when the buyer knows what it covers, which unit it uses, whether freight and tooling are included, whether the specification matches, whether the supplier is qualified, and what evidence supports it. Without that context, an apparently precise comparison can create false confidence.

I would therefore define the product as an **evidence-backed procurement decision system**. Its core responsibility is to establish comparability before optimizing price. It should separate quoted values from normalized values, distinguish eligible vendors from merely cheap vendors, flag missing or ambiguous information, and preserve the evidence behind every recommendation.

The longer-term opportunity is an **exception-resolution loop**: extract the quote, detect what prevents comparison, generate a targeted clarification request, update the offer when the supplier responds, and preserve the decision history. In other words, the product should not only reduce spreadsheet work; it should reduce the risk of making a high-value award decision on incomplete or falsely comparable data.

My prototype focuses on quote comparison because that is the clearest entry point, but the broader product is **extract** → **validate** → **clarify** → **compare** → **recommend** → **audit**.